Stagecraft Action Editor and Target Range Guide

Package version 1.0.0 | Environment Unity 2022.3 LTS, URP fully verified (see 1.1 for HDRP and Built-in) | Describes feature operation and the runtime integration contract, not implementation details

1. Installation

1.1 Prerequisites

- Unity Editor **2022.3.x LTS** (any patch of the 2022.3 channel) — the package targets `unity: 2022.3`; installing into 2019.4 / 2021 / 2023 will fail with assembly reference errors
- Render pipeline — the Action Editor itself works on any pipeline; the bundled Target Range scene has stricter requirements:

Pipeline	Action Editor	Target Range scene	Verification status
URP	Full support	Full support	Verified — the 1.0.0 release was built and tested on URP
HDRP	Full support	Works, but you must assign your own Target Dummy prefabs — the placeholder capsule that the scene spawns when no prefab is configured uses a Built-in shader and renders magenta under HDRP	Editor verified; Target Range runtime not exercised on HDRP
Built-in Render Pipeline	Full support	Not supported — scene setup relies on SRP features (shader graph fallback, post-processing volume)	Never supported

- The host project must have `com.unity.test-framework` in its manifest — the EditMode test assembly is compiled against `UnityEngine.TestRunner` and `UnityEditor.TestRunner`

Verify the last item in `Packages/manifest.json`:

```
{
"dependencies": {
"com.unity.test-framework": "1.4.6"
}
}
```

1.2 Install from a Local Folder (Recommended for First Use)

This is the standard flow when you unzip the release archive or clone the repository.

Step 1 — Place the package folder

Unzip `StagecraftActionEditor_1.0.0.zip` into any permanent location on disk, for example:

```
D:\UnityPackages\StagecraftActionEditor_1.0.0
```

The folder must contain these entries at its root:

```
package.json

README.md

LICENSE

Runtime/

Editor/

Tests/

Documentation~/
```

Do **not** place it inside the host Unity project's `Assets/` or `Packages/` folder.

Step 2 — Edit the host manifest

Open your host project's `Packages/manifest.json` in a text editor and add one line to the `dependencies` object:

```
{

"dependencies": {

"com.stagecraft.actioneditor": "file:D:/UnityPackages/StagecraftActionEditor_1.0.0"

}

}
```

Use forward slashes (`/`) or escaped backslashes (`\\`) — Unity accepts both on Windows. The rest of `manifest.json` does not need to change; other entries remain untouched.

Step 3 — Open / Re-open Unity

Launch your host project in Unity. The Package Manager will detect the new `file:` reference, compile the package, and create a cache entry under `Library/PackageCache/com.stagecraft.actioneditor@...`. This first compile typically takes 20-60 seconds; subsequent opens reuse the cache and are near-instant.

Step 4 — Verify the install

Once the editor window finishes loading, open the menu `Tools` in the top menu bar. You should see:

```
Tools

├─ Stagecraft

│    ├─ Action Editor
```

```
|   ├─ Target Range
|   |   ├─ Open Target Range
|   |   └─ Build Target Range Scene
|   └─ Action Editor Tests
|       └─ Run EditMode Tests
```

Open `Tools / Stagecraft / Action Editor` — a dockable editor window should appear. Confirm the window title says **Stagecraft Action Editor** and that preferences default to English (`Assets` for save path, no scan paths pre-configured).

Step 5 — Run the self-test (optional but recommended)

Choose `Tools / Stagecraft / Action Editor Tests / Run EditMode Tests`. A progress bar will appear in the Unity Editor window. When it finishes, a summary is printed to the Console. All 62 tests should pass — if any fail, paste the Console output to the issue tracker before proceeding.

1.3 Install from a Git URL

This install path is for customers who were granted private Git access after purchase. If you downloaded the package from the Unity Asset Store, use 1.2 instead — do not clone the public GitHub repository, which contains documentation only.

Add to `Packages/manifest.json`:

```
{
"dependencies": {
"com.stagecraft.actioneditor": "https://git.example.com/your-group/stagecraft-actioneditor.git"
}
}
```

Unity will clone the repository into `Library/PackageCache/` on first resolve. A Git URL install has two caveats:

- The repository **must** have a `package.json` at the root and the Git client (Git for Windows) must be on `PATH`
- If you need to pin a specific commit, append `#<commit-sha>` (or `#v1.0.0` for an annotated tag)

1.4 Uninstall or Upgrade

Uninstall — remove the single `com.stagecraft.actioneditor` line from `Packages/manifest.json`, restart Unity, and delete the package folder from disk if you also want the files gone.

Upgrade — overwrite the package folder contents with the new version (keeping the folder path unchanged, so your `manifest.json` line stays valid) and restart Unity. Alternatively, point `manifest.json` at the new folder path / new Git tag.

Migrating from the Chinese version (com.stagecraft.editor)

The English package uses a different identity (`com.stagecraft.actioneditor`, `Stagecraft Action Editor`) and its own preference file (`Editor/ActionEditor.txt`, instead of `Editor/Stagecraft.txt` in v2.1.10). The two packages can coexist in the same project for comparison, but sharing an action file between them may produce different default values on first load — this is expected.

1.5 Troubleshooting

Symptom	Likely cause	Fix
`error CS0246: The type or namespace name 'Stagecraft' could not be found`	Package reference not resolved	Check `manifest.json` — the `file:` path must point at the folder that contains `package.json`
Package resolves but menu does not appear	Installed into `Assets/` instead of the Packages system	Move the folder out and use `file:` in `manifest.json`
`Assembly 'Stagecraft.Editor' references 'UnityEngine.TestRunner'`	Missing test-framework package	Add `"com.unity.test-framework": "1.4.6"` to `dependencies`
`Action Editor window opens blank`	The editor assembly did not compile	Check the Console for `CS` errors first; the window draws nothing while the assembly is broken
Splitter minimum width seems too small	The project has a stale prefs file	Delete `Packages/com.stagecraft.actioneditor/Editor/ActionEditor.txt` (it will be regenerated with defaults)

1.6 Sample Content

The package ships two demo action files under `Samples~/Demo Actions`. They are not imported automatically — open the Unity Package Manager, select **Stagecraft Action Editor**, expand **Samples**, and click **Import** next to **Demo Actions**.

File	Event	Layout
`Melee Single Target.action.bytes`	Melee Single-Target (1)	Animation + Signal + Effect + Audio on a 1.0 s timeline; event at 0.45 s
`Melee AoE.action.bytes`	Melee AoE (3)	Same tracks on a 1.2 s timeline; event at 0.6 s with a 120° / 3.5 m sector

Both actions reference the neutral demo assets bundled in `Demo Assets/` next to them: the capsule `Demo Hero` prefab (role), a 1.0 s generic clip named `Demo Attack`, a 24-particle hit burst, and a synthesized 0.3 s hit sound. They therefore preview and play as soon as the sample is imported, with no asset assignment step. Replace any of them with your own content at will; the event clip independently carries the runtime contract (event ID, trigger time, and region parameters). If a path stops resolving — for example after a package version change or after moving the folder — run `Tools/Stagecraft/Samples/Repair Demo Asset Paths`, which relinks broken paths to the asset of the same name inside `Demo Assets/`; the same repair also runs automatically after import when a path does not resolve.

1.7 Support

- **Issue tracker:** https://github.com/Sam8164/stagecraft-actioneditor/issues
- **Source and release notes:** https://github.com/Sam8164/stagecraft-actioneditor
- **Documentation:** `Documentation~/` inside the package (this guide as PDF and plain text, plus EULA.txt)

When filing an issue, include the Unity version, the render pipeline, the exact menu path you used, and the full Console output. For asset problems, attach the action file (`.action.bytes`) that reproduces it — it is plain JSON and safe to share once you replace any project-specific asset paths.

1.8 Five-Minute Quick Start (Animators and Designers)

No contract knowledge needed. This walks one action from an empty file to a verified preview.

1. **Open the Editor** — menu `Tools/Stagecraft/Action Editor`.

2. **Create an action file** — click **Create (Default Path)** at the top; the file appears under your configured action folder with the `.action.bytes` extension.

3. **Add an animation** — right-click the Animation Track lane → **Add Clip** → pick your character animation. Drag the clip so it starts at 0.

4. **Add an effect (optional)** — right-click the track area → **Add Track** → Effect Track → **Add Clip** → drag in a particle prefab. Set the start time where the hit should land.

5. **Add a gameplay event** — **Add Track** → Signal Track → **Add Clip** → **Event**, then choose the event in the Inspector (Melee Single-Target, Melee AoE, Projectile, Instant Laser, Enemy Buff, Self Buff). Fill only the fields you understand; the rest keep working defaults.

6. **Preview** — scrub the playhead or press Play in the Editor's preview controls. Character animation, effects, and the event markers play in the Scene view.

7. **Save** — Ctrl+S. The file is written immediately; the Asset List shows it with a warning column that should stay empty.

8. **Verify in the Target Range** — menu `Tools/Stagecraft/Target Range/Open Target Range`, pick this action in the Action binding box, press **Run**, then check which dummies light up. Fill the **Sign-off Log** at the bottom when the result is right.

9. **Repeat** — change one thing at a time; the Target Range rereads the file on every run, so no reimport is needed.

Sampling the shipped examples is faster than starting empty: import `Samples~/Demo Actions` through Package Manager, open `Melee Single Target.action.bytes`, and follow steps 6–8.

2. Overview

This toolchain consists of two Unity Editor windows that cover the complete loop from authoring skill presentation to verifying it:

- Action Editor (the "Editor" below): opened from Tools/Stagecraft/Action Editor. Arranges animations, effects, audio, and gameplay signal events on a timeline and produces action files.
- Target Range: opened from Tools/Stagecraft/Target Range/Open Target Range, or launched by the "Launch Target Range" button in the Editor. Loads a character and target dummies into a controlled 3D scene, plays action files in real time, and verifies event detection, ranges, VFX, and hit feedback.

Action file: extension .action.bytes, content is JSON text, directly diffable and version-controllable. The file carries two kinds of data:

- Contract data: fields written into the file and consumed by the game runtime (such as event type, region shape and size, VFX paths, spawn point offset).
- Editor data (editorData): fields that only serve Editor preview and operation (such as preview projectile speed, attach point, playback speed); ignored at runtime.

2.1 Event Type Summary

Events (signal clips) are the only contract interface between action files and game logic. Event types are driven by the config file SignalConfig.json and are currently defined as follows:

ID	Name	Semantics	Contract fields (written to the action file)
1	Melee Single-Target (Hit)	Applies one damage instance to the nearest single target inside the attack range; the action only records the trigger time point, damage values are decided by the Skill Table	eventName
2	Projectile (EmitBullet)	Launches a projectile from the character; resolves on hitting the locked target	spawnOffset, projectilePath
3	Melee AoE (Aoe)	Applies one damage instance to every target inside the region, with region shape and size taken from the action file; may carry a Release VFX (played once at the region center)	regionShape, regionRadius, regionAngle, regionWidth, regionLength, regionOffset, hitEffectPath
4	Instant Laser (InstantSpell)	Performs an instant beam test against the target, with refraction supported; the action only records the trigger time point	eventName
5	Enemy Buff (AddBuff)	Applies effects centered on the locked target to	regionShape, regionRadius,

		enemies within a sector/rectangular region (Hit VFX + Persistent VFX)	regionAngle, regionWidth, regionLength, buffSectorDir, buffRectOffsetX, hitEffectPath, persistEffectPath
6	Self Buff	Applies a Persistent VFX to the character itself, following the character; casting again destroys the old one and creates a new one, with no hit test	persistEffectPath

Region shape (regionShape): 0 = sector (regionAngle = 360 means a full circle), 2 = rectangle. Melee AoE and Enemy Buff share the same set of region fields; the shape detection logic and display are completely identical in both places.

2.2 Track Type Summary

Track	Quantity constraint	Carried clips	Purpose
Animation Track (AnimationTrack)	Unique per group, cannot be deleted	PlayAnimation	Character skeletal animation; defines the action duration baseline
Effect Track (EffectTrack)	Unlimited	PlayParticle	Instantiates particle effects at character attach points
Signal Track (SignalTrack)	Unlimited	TriggerEvent (events 1-6), Tremble, Screen Shake, Flash	Gameplay events and camera/character presentation signals
Audio Track (AudioTrack)	Unlimited	PlayAudio	Audio; supports mute/solo preview

3. Action Editor

3.1 Window Layout

Opened from Tools/Stagecraft/Action Editor; minimum size 500×250. Five areas:

- Top bar: six playback control buttons (Go to Start, Previous Frame, Play/Pause, Stop, Next Frame, Go to End) + function buttons (Asset List, Launch Target Range, Save, Settings).
- Left track list: track headers and group management; the width is a draggable divider (default 180 px).
- Center timeline: time ruler, playhead, track clip area; mouse wheel zooms, middle button pans.
- Right Inspector (default 380 px, draggable divider): a fixed "Quick Selection" area at the top, with an asset-level or clip-level field panel below.
- Bottom bar: "Last save time" on the left; "Clear Copy" and "Clear Selection" link buttons on the right.

Note: the window title has no unsaved asterisk; the unsaved state is conveyed by confirmation dialogs when switching action/character or closing the window.

3.2 Standard Workflow (Starting from Scratch)

Step	Operation	Description
1	Set paths	On first use, configure the prefab scan path and the action save path in "Settings" (both must be inside the project Assets).
2	Select character	Choose a character prefab from the "Character" dropdown in the Inspector quick area; a preview model named [Preview]xxx appears in the Scene view (does not pollute the scene).
3	New action	Enter a file name in "New Action" in the quick area → create; the current character is bound automatically and the file opens. Duplicate names (case-insensitive) are rejected.
4	Arrange the timeline	Drag in animation assets; place signals with the in-row buttons on a Signal Track (Event/Tremble/Screen Shake/Flash) or the "Add Clip" context menu; drag in particle/audio assets to create the corresponding clips.
5	Edit fields	Select a clip and edit the event type, region shape and size, VFX paths, etc. in the Inspector; the Scene view shows range helpers live.
6	Save	Ctrl+S or the "Save" button at the top; empty tracks are stripped automatically.
7	Verify in the Target Range	Click the green-outlined "Launch Target Range" button at the top (prompts first if unsaved).

3.3 Inspector · Quick Selection Area

- Prefab paths: the list of scan directories for character prefabs; "＋" adds (must be inside Assets), "—" removes the current entry.
- Action paths: the list of save directories for action files; managed the same way.
- Character dropdown: lists the scanned character prefabs. If the action has no bound character, selecting one writes the binding and saves automatically; if already bound, switching prompts an unsaved confirmation first.
- Action dropdown: lists .action.bytes grouped by save path and character; switching prompts an unsaved confirmation first.
- Used VFX: read-only text summarizing the asset file names of all VFX clips in this action.
- New Action: file name input (invalid characters filtered live) + Create button; validates empty name, invalid path, and duplicate name. When no character is selected, you are prompted to select one first; the new action belongs to it automatically.

When a clip is selected, the Inspector first shows the clip name in 25 pt large text (format "Type · Name", e.g. "Event · Melee AoE"), then the field panel. Field edits within a 2-second window are merged into a single undo record.

3.4 Action File Management and Asset List Window

Save: Ctrl+S or the "Save" button at the top; JSON is serialized back to disk, empty tracks are stripped, and the save time at the bottom refreshes.

Asset List window ("Asset List" button at the top, minimum 860×320):

- Scan scope: recursively enumerates .action.bytes with the "Action save path" as root (falls back to Assets/Bundles if the path is invalid); rescans automatically every 0.5 s by directory fingerprint.
- Toolbar: refresh scan, root directory display, signal filter dropdown ("All" + each event "ID Name", deprecated events marked "Deprecated"), file name search box (case-insensitive), asset count.
- Table columns: Character, Asset file, Track count, Clip count, Signal summary (deduplicated), Warning count (yellow), Warning details (deprecated/undefined signals, missing assets, empty action, parse failure).
- Interaction: click to select, Ctrl+click to multi-select, left-button drag beyond 4 px for box selection, double-click to open (unsaved confirmation first), context menu delete, Delete key delete, Ctrl+Z undo delete (restores the whole batch, up to 100 batches, invalid after domain reload).

3.5 Track System

- Group: currently a single-group structure; the group row contains a fold arrow and the group name; old data group names migrate automatically to "EffectN". Group menu: Add Track (Signal/Effect/Audio; Animation Track is unique and therefore grayed out), Paste, Delete.
- Track header: fold arrow (a Signal Track expands a clip navigation sub-row; multiple Signal Tracks show indices), color bar, type icon, name (Audio Tracks show the audio name of the first clip, with "+N" for multiple audios).
- Signal Track in-row shortcut buttons: Event, Tremble, Screen Shake, Flash (28×16).
- Audio Track M/S buttons: Mute (red), Solo (orange); effective in Editor preview only and not written to the action file.
- Track deletion: the "—" button at the far right of the header (Animation Tracks cannot be deleted).
- Disable/Lock: a diagonal stripe overlay and Disable/Locked stamps; right-click menu on the left side of a track: Disable, Locked, Copy, Cut, Delete.
- Reorder: drag up/down in the left area of the track; the target row shows a white underline and a "Move Track" undo record is produced.
- Clip navigation sub-row: click to jump the playhead to that clip's time and scroll it into view.

3.6 Clip Operations

Three ways to add clips:

- Signal Track in-row buttons: "Event" opens the event picker window (EventPickerWindow, 400 wide, event names bold + gray descriptions, deprecated entries grayed out and not creatable, blocked entries hidden, single click creates with no confirmation); Tremble/Screen Shake/Flash are created directly at the snapped playhead time. If a signal already exists at the same time on the same track, the action is rejected with "Duplicate signal at the same time on the same track. Please create a new track".
- "Add Clip" context menu on empty track space: lists attachable types; PlayAnimation is skipped when an animation already exists; animation clips are forced to time 0.
- Dragging assets in from Project: an animation asset replaces the asset of the selected or first animation clip ("Replace Animation Asset" undo); other assets create a clip at the drop time and take the asset's original length; on overlap with existing clips they are pushed to the end of the conflict chain; same-type tracks can be created automatically ("Drop Clip" undo).

Editing clips:

- Move: drag the clip body with the left button; a multi-selection moves as a whole keeping relative offsets; animation clips are locked at 0; overlapping drop points roll back with a warning.
- Resize: with a single selection and a resizable clip, the left/right edge hot zones show a stretch cursor; minimum 0.1 s, limited by adjacent clip boundaries; the left edge of animation clips is locked; stretching animation/audio recalculates playback speed so the content always fills the slot length.
- Drag assistance: during dragging, start/end guide lines (white solid when snapping hits, otherwise gray dashed) and time labels are shown; dragging outside the visible range shows "Zoom out with the mouse wheel to continue dragging" at the bottom.
- Delete: Delete key (not triggered when a text field has focus) or the context menu.
- Clipboard: Copy/Cut/Paste/Delete in the context menu; Ctrl+V pastes at the playhead time onto the selected track (if no track is selected, the track of the selected clip, otherwise the source track); pasted animation clips are forced to 0; pending paste content flashes on tracks/clips at a 0.5 s period.
- Multi-select: box select by right-button dragging in the track area (>10 px).
- Undo/Redo: Ctrl+Z / Ctrl+Y, covering add, delete, move, and property changes.

3.7 Timeline Interaction

- Time ruler: the color bar at the top indicates total length; in seconds mode the tick interval adapts to the visible width from 0.01 to 50 seconds; in frame mode frame numbers are shown at the configured frame rate.
- Playhead: white vertical line + icon at the top; clicking/dragging the time ruler moves the playhead, clamped within (0, total length), after which it enters a frozen preview state where playback stays paused.
- Total duration: not directly editable; recomputed on every repaint as the maximum end of all active clips (minimum 0.1 s).
- View range: viewTimeMin/viewTimeMax persist with the action file (default 0–5 s, minimum span 0.25 s, maximum 240 s); mouse wheel zooms, middle button pans (lower bound clamped to 0).

- Snapping: aligned to the snap interval; magnetic snapping is on by default and snaps to 0, total length, the playhead, and other clips' start/end points (10 px radius; hold Shift to disable temporarily).
- Playback control: Go to Start, Previous Frame (steps by the snap interval and wraps around), Play (cyan highlight while playing), Pause, Next Frame, Go to End.

3.8 Inspector · Event Clip Fields

A signal clip always shows the "Event Type" dropdown at the top (driven by SignalConfig; deprecated entries marked, blocked entries hidden). When unconfigured, the clip shows "Event · Unconfigured" in red. Asset-type fields are object boxes; clicking opens the asset picker (AssetPick) or you can drag in directly from Project.

Event 1 · Melee Single-Target (Hit)

No extra fields. The action only records the trigger time point.

Event 2 · Projectile (EmitBullet)

Field	Type/Default	Description
spawnOffset Spawn Offset	Vector3, default (0,1,0)	Local coordinates of the character root; follows rotation but not scale; written to the contract
projectilePath Flight Effect	GameObject	Projectile prefab; written to the contract
projectileSpeed Projectile Speed	default 3 m/s	Editor preview config only (projectilePreviews)
projectileDistance Range	default 10 m	Editor preview config only
parabola Parabola	toggle	Editor preview config only
parabolaHeight Parabola Height	default 2 m	Editor preview config only

Event 3 · Melee AoE (Aoe)

Field	Type/Default	Description
regionShape Shape	0 sector / 2 rectangle	Written to the contract; the game determines the damage region from it
regionRadius Radius	default 3 m	Sector/full circle; written to the contract
regionAngle Angle	default 360°	360 means a full circle; written to the contract
regionWidth Width	default 2 m	Rectangle, symmetric left/right about the facing axis; written to the contract
regionLength Length	default 4 m	Rectangle, extending along the facing axis; written to the contract
regionOffset Region Center Offset	Vector2 (X/Z)	Character-local horizontal offset, used so VFX fit the action; written to the contract
hitEffectPath Release VFX	GameObject	Played once at the region geometric center when the event triggers (rectangle = anchor advanced another length/2 along forward; sector/full circle = circle center); written to the contract; Editor preview/Target Range/game all use

		the same play point

The division of labor between the Release VFX slot and the Effect Track (PlayParticle): the Release VFX slot is driven by event semantics, and its play point automatically follows the region geometric center, changing whenever the region parameters change — suitable for the burst/ring of "this AoE hit" without manual placement; the Effect Track is a general art layer whose position/rotation/scale can be hand-placed, stacked in multiple layers, and parented to the character root — suitable for slash arcs, trails, and decoration unrelated to events. The two solve different problems and cannot replace each other; do not configure the same VFX in both places, otherwise it plays twice. Slight offsets caused by the prefab's own pivot/visual center of gravity are not compensated by the tool — for fine adjustment, hand-place it with an Effect Track, or adjust the prefab pivot directly.

Event 4 · Instant Laser (InstantSpell)

All Inspector fields are Editor preview configuration (laserPreviews); the real runtime length = attack range:

Field	Type/Default	Description
length Laser Length	default 6 m	Editor preview only
pierce Pierce	toggle	Editor preview only
refract Refraction Segments	integer	Editor preview only
laserEffectPath Laser Effect	GameObject	Beam prefab
originOffset Origin Offset	default (0,1.5,0)	Editor preview only

Event 5 · Enemy Buff (AddBuff)

The region is centered at the feet of the locked target, and the direction reference = target→hero line. Shape switching decides which fields are visible:

Field	Type/Default	Description
regionShape Affected Shape	0 sector / 2 rectangle	Written to the contract
regionRadius Radius	default 3 m	Sector; written to the contract
regionAngle Angle	default 360°	Sector aperture; ≥360 means a full circle; written to the contract
buffSectorDir Sector Direction	degrees, default 0	Relative to the target→hero line, clockwise positive; 0 = opening faces the hero, ±180 = facing away; written to the contract
regionWidth Width	default 2 m	Rectangle; written to the contract
regionLength Length	default 4 m	Rectangle; written to the contract
buffRectOffsetX Forward/Backward Offset	meters, default 0	Offset along the target→hero line, positive = toward the hero; written to the contract
hitEffectPath Hit VFX	GameObject	Created at the hit point on hit; written to the contract
persistEffectPath Persistent VFX	GameObject	Loops during the buff, destroyed and recreated on re-hit; written to the contract
hitOffset Hit Point	default (0,1,2)	Editor preview config only (buffPreviews); shown as a pink-purple marker in the Scene view
persistOffset Persist Point	default (0,1.8,0)	Editor preview config only; shown

		as a white marker in the Scene view

Legacy data compatibility: while the new field regionRadius is still at its default value, the historical field buffRadius migrates automatically.

Event 6 · Self Buff

Field	Type/Default	Description
persistEffectPath Persistent VFX	GameObject	Loops during the buff and follows the character; destroyed and recreated on recast; written to the contract

Attach point offset is deprecated: Self Buff VFX are presented through Effect Tracks, and the in-event offset field is no longer used.

3.9 Inspector · Animation/Audio/Effect Clips

- PlayAnimation: animationClip object box (an AnimationClip can be dragged in), speed playback speed (default 1). Speed and slot length are linked: changing speed automatically adjusts the slot length to "clip length/speed", and stretching the slot length recalculates speed. There are also loop and mirror preview toggles (Editor only).
- PlayAudio: audioClip object box, volume (0–5 slider, default 1; values above 1 are gain-baked in preview), speed (0.1–3, linked to slot length), fadeIn/fadeOut in seconds. A cyan waveform is drawn on the track.
- PlayParticle: resPath VFX prefab (GameObject), position/rotation/scale offsets. During preview, particles are instantiated under the preview character and the simulation is advanced manually.

3.10 Inspector · Tremble/Screen Shake/Flash Signals

All three are zero-length time-point signals that only record the trigger moment and presentation parameters, previewed in the Editor when the playhead passes over them:

- Character Tremble (TriggerCharacterTremble): amplitude (default 0.03 m, the upper limit of random root-node offset), duration (default 0.2 s).
- Screen Shake (TriggerScreenShake): amplitude (default 0.15 m), duration (default 0.25 s, sinusoidal reciprocation over 3 cycles).
- Flash (TriggerFlash): color (default white, instantaneous tint over the whole character), duration (default 0.3 s; one event flashes once — copy the clip for multiple flashes). Changing a parameter replays the preview immediately.

3.11 Scene View Helpers and Preview

When a clip is selected or playback passes over it, helper graphics are drawn in the Scene view:

- Melee AoE (AoePreview): an orange range with the character as origin — a sector is a translucent solid arc plus two radial edges, a rectangle is a translucent polygon; a draggable spherical handle sits at the center offset for direct editing of regionOffset. At the event time the Release VFX is also played at the region geometric center (with a 4-second fallback destroy), the

same play point as the Target Range/game, making it easy to check whether the VFX lands at the center of the frame.

- Enemy Buff (BuffEffectPreview): a pink-purple disc marker at the hit point and a white marker at the persist point (both with a black outer ring).
- Instant Laser (LaserPreview): the main beam is bright cyan, pierce segments the same color but slightly faded, refraction segments a warm-yellow polyline chain; a spherical handle sits at the origin.

Preview mechanism: after selecting a character, a preview model [Preview]xxx is created in the Scene view (HideFlags.DontSave, does not pollute the scene or the undo stack); the view uses Unity's default Scene view orbit/zoom. During playback/playhead dragging, animation is sampled by time (including root-node translation); particles are simulated frame by frame; audio is played at varying speed through the preview AudioSource; Tremble/Screen Shake/Flash are self-advancing previews. Pausing, stopping, or switching the action or character all restore the scene uniformly.

3.12 Preferences ("Settings" Button at the Top)

- Language; asset picker style (Tiled/Folder).
- Time step mode: seconds (snap interval 0.001/0.01/0.1 selectable) or frames (frame rate 24/25/30/60 selectable).
- Magnetic snapping toggle (on by default).
- Action save path: read-only box + "Select" button.
- Color configuration: tabs by asset type, coloring per track and per clip type.

Settings take effect immediately; clicking "Save" writes to the in-package config file `ActionEditor.txt`.

3.13 Event Config File SignalConfig.json

Located at Editor/Config/SignalConfig.json inside the package; it defines event entries (value ID / name / desc / deprecated / blocked / note). It uniformly drives: the event type dropdown, the event picker window, the Asset List signal filter and signal column, and clip display names ("Event · ×××"). Changes are detected by version number and refresh the cache. Deprecated events cannot be created but old clips still display; blocked events do not appear at all.

3.14 Shortcuts and Context Menu Summary

Scenario	Operation	Behavior
Main window	Ctrl+Z / Ctrl+Y	Undo / Redo
Main window	Ctrl+S	Save the action file
Main window	Ctrl+C / Ctrl+X / Ctrl+V	Copy / Cut / Paste (paste at the playhead)
Main window	Delete	Delete the selected clip (not intercepted when a text field is focused)
Clip context menu	Copy/Cut/Paste/Delete	——
Empty track context menu	Add Clip	Lists attachable clip types
Track left context menu	Disable / Locked / Copy / Cut / Delete	——
Group row context menu	Add Track / Paste / Delete	——
Asset List	Double-click / Delete / Ctrl+Z / right-click delete	Open / Delete / Undo delete

Platform note: shortcuts use **Ctrl**. On macOS, Unity maps the Control key to `Event.control`, so Ctrl+Z / Ctrl+S / Ctrl+C / Ctrl+X / Ctrl+V also work there; the **Command** key is recognized in the Asset List only. Windows is the verified platform; macOS has not been tested.

4. Target Range

4.1 Opening and Window Layout

Menu Tools/Stagecraft/Target Range/Open Target Range, or launched by the "Launch Target Range" button at the top of the Editor (no parameters are passed to the Target Range). Window minimum 1120×680.

- Top bar: title, "Create New Target Range" button, "Minimize" button; when minimized it collapses to a 320×30 horizontal bar (showing "Target Range (Running)/(Minimized)" + "Restore"), and running is not interrupted.
- Left render preview area: an off-screen render image, centered by viewport aspect ratio, with the current resolution labeled at the bottom left (e.g. 1080×1920). The preview camera does not accept mouse rotation/zoom: its position is always "character position + cameraOffset" with smooth following, and it always looks at a point 1 m above the character.
- Center dynamic area (330 px, titled "Dynamic Area · Live-editable at runtime"): formation, target dummy Hit Radius, target dummy scale, camera, attack range, invocation items, hit slots, move speed, playback control, direct event parameter editing (blue frame).
- Right static area (350 px, scrollable, titled "Static Area · Configured before running"): target range file, viewport aspect ratio, character, action assets, target dummies, render pipeline; the whole area is locked and grayed out while running (except the action asset import and target range file areas).

4.2 Target Range File Management

- Create: the first item in the static area, "Target Range File", contains "Storage Path" (text box + "…" browse; a folder inside the project Assets must be chosen, otherwise a dialog rejects it) and "Target Range Name". Creation is complete only when both are non-empty; otherwise all other features are grayed out with a prompt. Invalid file name characters are replaced automatically with underscores; clearing either item and submitting shows "Cannot be empty" and reverts. Default storage path Assets/Bundles/Stagecraft_TargetRange/Ranges.
- Rename/Change path: uses "write new, delete old" — the config is first saved at the new location, then the old file and its .meta are deleted and the list refreshed.
- Target range list (N): a collapsible panel listing all .json target ranges in the current path (sorted by name), with the current item highlighted in blue; left-click opens it (stops running, loads, rebuilds the preview); the only right-click menu item is "Delete", with a second confirmation. Deleting the currently open target range continues automatically: open the next one (if it was the last, open the previous one; if none remain, return to the uncreated blank mode and clear the invocation items and the selected action).

- Legacy migration: when the window opens, if the legacy fixed-path file Assets/Bundles/Stagecraft_TargetRange/Config/target_range.json is found, it is migrated automatically to Ranges/Default Target Range.json (preserving the GUID preferentially) and loaded.
- "Create New Target Range" button: returns to blank creation mode.

4.3 Static Area Configuration Details

- Viewport aspect ratio: a dropdown of 8 presets (Small phone 720×1280 / Mainstream 1080×1920 / HD 1440×2560 / Ultra-tall 1440×3040 / iPad 1536×2048 / Tablet landscape 2048×1536 / 1080p / 2K) + custom width×height (each clamped to 240–4096). Affects only the render area's aspect ratio; it is not displayed at true pixel size.
- Character prefab: object box (shows the asset name, shows the path on hover, accepts a dragged GameObject). Changes are saved immediately and the preview rebuilt; if left empty, no character is spawned (the scene stays empty).
- Character Idle / Character Move: dual-binding controls — "Action/Animation" radio + asset field + "×" clear. Clicking the asset field opens a file dialog (action .action.bytes / animation .anim) and also accepts drops from Project or from the action buttons in the dynamic area. The row of the inactive source is grayed out at 45% opacity but still clickable to switch; filling in an action switches to the action source automatically, and filling in an animation switches to the animation source automatically. After binding, a loop check runs: a warning is shown when the asset cannot be resolved or does not loop (warning only, not blocking).
- Action assets (available while running too): two buttons "Select File" and "Select Folder" (recursively scans *.action.bytes) that import into the dynamic area's invocation item list, up to 30.
- Target dummy prefab: object box. If left empty, no target dummies are spawned; only when a prefab is specified but fails to load does it fall back to a red capsule placeholder.
- Target dummy idle: the same dual-binding control.
- Target dummy hit (mutually exclusive, may be left empty; target dummies do not check loop): fixed 3 hit slots, each with two mutually exclusive rows for action/animation (filling one clears the other, and both may be empty). After a binding change, if the currently active slot becomes invalid, the first slot with a binding is activated automatically.
- Render pipeline (material compatibility): four options — Auto-detect (recommended) / Built-in / URP / HDRP; only affects materials of procedural objects (ground, range rings, marker lines); switching rebuilds immediately and "Currently active: ××" is shown below. When an explicit option conflicts with the actual current pipeline, the actual pipeline wins (shaders from the other pipeline render as purple blocks in this pipeline); the final fallback only borrows the shader of the character material and never copies character textures, so the ground/range rings will not show the character atlas.
- "▶ Run Target Range" button: enters the running state and locks the static area.

4.4 Dynamic Area Operation Details

- Target dummy formation: three options Single / Row / Triangle (switchable only while running; switching rebuilds the scene and automatically resumes running). Single = one dummy 3 m

straight ahead; Row = three in a row, spacing = spacing distance; Triangle = equilateral triangle, side length = spacing distance. When not Single, a "Spacing Distance" numeric box is shown (0.5–10 m, default 2).

- Target dummy Hit Radius (m): directly below the three formation buttons, 0–5, step 0.05, default 0.5. The value is configured for the **current target dummy prefab** and written to the target range config file (prefab path → radius mapping, plus a global default); dragging refreshes all on-scene dummies of that prefab immediately and recalculates hits, and the value is persisted on mouse release. Switching the prefab or switching the target range shows the value already stored for that prefab.
- Target dummy scale: same position, directly below the Hit Radius, multiplier 0.5–3.0, step 0.05, default 1.00, stored in the target range config as `dummyScale`. The scale is computed as "prefab original scale × multiplier" (not cumulative), the Hit Radius grows automatically with the scale, and a small mob can be enlarged into an elite. The multiplier is currently uniform for the whole scene; per-prefab scaling within one target range is not supported.
- Target dummy hit ring: at runtime each dummy shows a light-cyan thin ring at its feet, radius = that dummy's effective Hit Radius (configured value × instance scale); dragging any slider updates it immediately, and it hides automatically when running stops. Together with the orange range frame it expresses the detection semantics, see 4.7.
- Camera position (follow-locked · world offset): X/Y/Z three boxes; hovering over a label allows left/right dragging to adjust the value (0.02 per pixel), and Y may not be negative; default (0, 30, -7). The target point is always locked to the character and followed smoothly, and screen shake is layered on top of this.
- Attack range (m): default 1; changes update the detection and the blue range ring immediately (clamped to 1–6 m when the config loads).
- Actions (click to switch at runtime): that is, the invocation item list (up to 30). Buttons are sorted by file name (with the .action.bytes suffix removed) and laid out in 1–3 columns adapting to the width; each cell = a name button + a "−" delete button. The selected item uses the raised button style. Clicking a name toggles the selection: switching first hard-stops and clears all effects and audio in the scene (ClearAllSceneEffects), then persists the selected action. Holding the name button allows dragging it into the static area's "Action" binding box. During a single playback the whole list is grayed out. An empty list shows an import prompt; importing more than 30 at once is rejected as a whole with a dialog.
- Target dummy hit (mutually exclusive): 3 equal-width yellow highlighted buttons; the text prefers the action file name, then the animation file name, and shows "(Empty Slot N)" when both are empty; lit buttons have a yellow background (RGB 1,0.8,0.25), empty slots are grayed out; clicking toggles between lit and unlit and persists immediately.
- Character move speed: default 1 m/s (minimum 0.1); when binding an action, the action's moveSpeed is written at the same time.
- Playback control: a status line (e.g. "Target Range running · action loop: xxx", "Single playback: xxx", "Target Range ready · running mode: none") + a "▶ Play Once" button (clickable when an action is selected and the Target Range is ready). Single playback interrupts everything currently running, plays the selected action once, and automatically returns to the idle loop when it finishes.

- "■ Terminate Target Range" button (only while running): stops running and rolls the character position and rotation back to the pre-run state; two lines of diagnostic text are shown below (preview/pipeline/character/dummy count/range ring state; camera and character coordinates).

4.5 Direct Event Parameter Editing (Blue Frame)

When the selected action is non-empty, the dynamic area shows the blue-outlined panel "Event Parameters · {file name}"; this is direct editing of the selected .action.bytes file, and every change is written back to the file immediately:

- Animation group: each animation clip shows "Action Duration X.X s" and "Playback Speed" (0.1–5). After changing the speed, signal trigger times exceeding the duration are clamped back to the duration value and the notification "Event trigger time exceeds the animation duration" is shown.
- Signal group: collapsible groups in the order ①Melee Single-Target ②Projectile ③Melee AoE ④Instant Laser ⑤Enemy Buff ⑥Self Buff (titles carry "· N clips"). Each clip allows editing "Trigger Time (s)" (0–60; values beyond the animation duration are rejected and the same notification is shown).
- Projectile additionally allows editing: spawn point offset, flight effect, flight speed (minimum 0.1), parabola toggle and parabola height (0.1–20).
- Melee AoE additionally allows editing: shape (sector/rectangle), radius/angle or width/length, range center offset (X/Z).
- Instant Laser additionally allows editing: spawn point offset, laser effect (length is not offered in the Target Range; the real length = attack range).
- Enemy Buff additionally allows editing: Hit VFX, hit point height, Persistent VFX, persist point height, affected shape; when the sector angle is <360 a "Sector Rotation (0 = facing hero)" appears, and for a rectangle a "Forward/Backward Offset (positive = toward hero)" appears.
- Melee Single-Target has no extra fields; Self Buff only has Persistent VFX and persist point height.

4.6 Running, Movement, and Camera

- Movement: the UI text refers to the Arrow Keys (↑↓←→). Holding the arrow keys moves the character relative to the camera's facing (speed = move speed), and the character smoothly turns toward the movement direction. On the action path, pressing switches to the move action and releasing switches back to the idle action (looping); on the animation path, the move/idle animations are sampled and switched.
- During skill animation playback, movement and turning are frozen (effect/audio tails are not frozen), and both resume immediately when the animation ends.
- Attack range blue ring: a cyan 64-segment ring, redrawn every frame following the character.
- Camera: fixed-offset following, no manual rotation or zoom.

4.7 Runtime Detection and Presentation of the Six Event Types

The playback engine has forward-only timeline semantics (the whole action is scaled by the playback speed of the first animation clip). Only when the action contains lock-semantic events (Melee Single-Target/Projectile/Enemy Buff) does playback start by instantly turning the character to face the locked target. Target locking refreshes every frame: the target dummy with the smallest horizontal

distance within the attack range; ties keep the old lock; when there is no target in range the lock is released.

- Melee Single-Target: detection is the intersection of the attack range and the dummy's hit circle; with multiple targets the nearest is chosen, and equal distances (≤0.01 m) are resolved randomly. On a hit, a hit is dispatched according to the hit slots; outside the range nothing is hit. No damage numbers; Console logs are output (e.g. "Melee Single-Target@0.5 hit=Y(dummy#2, candidates=2, distance 1.80m)").
- Projectile: spawn point = character root position + character rotation × spawn offset, and the flight effect is instantiated. With a lock: flies toward the target's chest (root + 1 m), maximum flight distance = distance to the chest + 2 m; without a lock: flies straight forward and is destroyed once it leaves the attack range. Parabola mode flies along 4h·s·(1-s) and lands exactly at the chest, with the effect's orientation fitted to the trajectory tangent every frame. A distance to the target's chest smaller than max(0.5, per-step displacement) counts as a hit, dispatches a hit, and the projectile is destroyed afterwards.
- Melee AoE: detection center = character root + character rotation × range center offset. Rectangle: with straight forward as the axis, |localX| ≤ width/2 and 0 ≤ localZ ≤ length; sector: distance ≤ radius and angle to forward ≤ angle/2 (360° means a full circle). Detection counts "the region intersects the dummy's hit circle": the distances/boundaries above are all expanded by that dummy's effective Hit Radius, i.e. distance ≤ region radius + hit radius. All dummies within the range are dispatched a hit; no locking, no turning. While running, if the selected action contains this event, an orange range frame is drawn around the character (sector: two radial edges + arc / rectangle: a five-segment line / full circle: 48 segments), and the blue attack range ring is hidden at the same time. The range frame only expresses the intended design range and no extra "expanded dashed line" is drawn; actual coverage is expressed by the intersection of each dummy's light-cyan hit ring with the frame. If the action file has a Release VFX configured, it is played once at the region geometric center (rectangle = anchor advanced another length/2 along forward, sector/full circle = circle center) with a 4-second fallback destroy; the Console outputs "AoE play point shape/anchor/geometric center" for easy verification.
- Instant Laser: origin = character root + rotation × spawn offset (same source as Projectile). Without a lock: fires a segment straight forward, endpoint = character root + forward × attack range (keeping the spawn point height). With a lock: the beam stops 0.35 m in front of the main target's chest and dispatches a hit; with refraction on (default on), a segment is refracted from the main target's chest to each other dummy in range with a separate hit dispatch each (in a Row formation, refraction is effectively penetration). Beam meshes are stretched exactly along the longest axis, and each segment is destroyed after 0.45 s.
- Spawn point visualization: while running and the current action contains Projectile/Instant Laser events, a red circle (radius 0.2 m, always facing the camera) + an orange line are drawn (connecting to the target's chest when locked, otherwise to the edge of the attack range ring).
- Enemy Buff: centered at the feet of the locked dummy (this event has no effect if there is no lock), with the direction reference = target→hero line. A sector rotates the direction reference by "Sector Direction" and uses it as the central axis for radius/aperture detection (≥359.5° is a full circle); a rectangle is offset forward/backward by "Forward/Backward Offset" from the target center along that direction and then tested by half width/half length. Dummies in range: hit

dispatch + red flash for 0.15 s + Hit VFX created at the hit point (4-second fallback destroy) + Persistent VFX created at the persist point; when the same dummy is hit again, its Persistent VFX is destroyed and recreated; the old Persistent VFX of dummies not hit this time is cleared. Visualization: an orange lock line (character chest→target chest), pink hit point and white persist point cross rings on the locked dummy, and an orange affected-range frame.

- Self Buff: the target is always the character itself; the Persistent VFX is parented to the character and moves with it; recasting destroys the old one and creates a new one; there is no hit detection at all.
- Hit slot playback rules: if the active slot is bound to an action, that action plays fully (its internal Flash/VFX and other events all take effect, and it returns to idle automatically when finished); if bound to an animation, it is sampled and played once; an empty slot is ignored. Idle also has both action/animation paths.
- Tremble/Screen Shake/Flash runtime behavior: random trembling of the character root node (mostly upward, linear falloff), sinusoidal screen shake of the camera (fixed 3 cycles), whole-character color flashing; effects of the same kind are mutually exclusive.

Note: the three config items enemyBuffHeight, regionCenterMode, regionForwardDistance are currently not consumed by detection — hit/persist attach points actually use the hit point/persist point of buffPreviews, and the region center is always the locked dummy.

4.8 Persistence and Refresh on External Modification

- Target range config is saved on change: every change to prefabs, dual bindings, hit slots, active slot, formation, spacing, camera, attack range, move speed, viewport, pipeline, selected action, invocation items, and target range name/path is written immediately to Storage Path/Target Range Name.json (not written to disk when not yet created).
- Blue-frame edits are written back to the action file immediately (UTF-8 without BOM) and invalidate the parse cache.
- After the action file has been modified externally (by the Editor): every load of the Target Range (single playback, entering the running state, moving and switching actions) rereads it from disk, so the next playback uses the latest content; the loop-check cache expires automatically based on the file write time. The blue-frame in-memory model is cached by path, but before each repaint external changes are detected by file write time and an update is reloaded automatically (with a Console notice), preventing an old model from writing the whole file back over what the Editor just saved when editing fields.

4.9 Play Mode Runtime Panel and Smoke Test

- Menu Tools/Stagecraft/Target Range/Build Target Range Scene: generates a scene file containing only the bootstrap components, used for Play Mode verification on device. The scene records the Target Range config open in the window; Hero Prefab Path and Actions Dir override it when filled.
- Runtime panel (TargetRangePanel): 1 skill button by default; "+ Add Skill Button" can add up to 8; the button text takes the action display name, "◀"/"▶" cycle the binding through the action directory, and "Cast" plays that action. The panel also has an idle action selector (choosing "None" stops the loop), three dummy formation buttons, an Enemy Buff height slider (0–3 m),

an attack range slider (1–6 m), and a laser refraction toggle; changes are written back to the config on mouse release.

- Smoke Test: in Play Mode, if ae_range_smoke.flag exists at the project root, all actions in the action directory are played in sequence, a ae_range_smoke.txt report is written, and Play Mode then exits; otherwise manual mode is entered.
- Sign-off Log (the "Sign-off Log" block at the bottom of the blue frame): after at least one playback, fill in Pass/Fail, Reviewer, and Notes, then click "Record This Sign-off" to write into StagecraftVerify/records.jsonl at the project root. Each record binds: the action file's relative path, the first 16 characters of its sha256, file size and modification time, toolchain version, dummy coordinates and Hit Radius, a snapshot of Target Range detection parameters, and the signals and hit list of this playback. Once the action file changes, the record is automatically marked "Stale" and must be signed off again; the ✓ / ✗ / ⟳ / — after each action list button means "Signed Off/Failed/Stale/Not Signed Off". The smoke report answers "does it run", the Sign-off Log answers "does art accept this attack-defense pairing"; the two are independent and do not replace each other.

4.10 UI Text Consistency

- There are currently no known discrepancies between UI text and actual behavior: the action list is labeled "Saved with the Target Range config", the target dummy prefab is labeled "not spawned when unspecified; the capsule is used only when loading fails", and the movement key text is "Arrow Keys"; all are consistent with the implementation.
- If any text/behavior inconsistency appears later, it is recorded uniformly in this section and cleared before delivery.

5. Runtime Integration

This chapter describes the complete integration contract from action files to the game runtime: file structure, mandatory fields, time semantics, event IDs, and currently verified inconsistencies. It can serve as the single source of truth for integrators or reviewers.

5.1 File Structure

- Two-part text: line 1 is the fully qualified asset type name (the Editor writes Stagecraft.SkillAsset; the game deserializes with an explicit type, so this line is only a marker), and from line 2 onward it is compact JSON (no indentation).
- Root keys include Temps, length, viewTimeMin, viewTimeMax, formatVersion, editorData. Among them groups is marked as not serialized; the whole hierarchy is carried by Temps.
- Temps is a container array of "type name + child JSON": Asset.Temps holds Group, Group.Temps holds Track, Track.Temps holds Clip. The reader must reflectively resolve the concrete class from type, then parse its json, restoring and rebuilding the hierarchy layer by layer.
- After restoration, Validate must be called once: recompute length (the maximum end of all effective clips, minimum 0.1 s), clean up parse results, and correct animation clip start points.
- Reflective restoration depends on the types not being stripped; projects using HybridCLR must keep the relevant types in link.xml.

- The encoding must be UTF-8 without BOM; with a BOM the type name on the first line contains invisible characters and parsing by type name fails.
- formatVersion is currently 2; missing is treated as 0 (initial version) and migration runs on read; a version higher than supported should raise a warning rather than being parsed silently.
- JSON key names map one-to-one to field names (lowercase first letter, e.g. startTime, length, active, resPath, eventName, regionShape); there is no extra wrapper layer, so readers can take values directly by field name.

5.2 Fields Actually Read by the Game Side Today

The table below states what an integrator is expected to consume and what the reference integration already handles. "Status" describes the reference path; a host implementation may differ, but it must honour the same contract.

Clip	Field	Purpose	Current status
PlayAnimation	resPath	Uses the file name to drive character animation playback	Read
PlayAnimation	startTime	Triggers once when this time is reached	Read
PlayAnimation	playbackSpeed	In-action speed change	Not read at runtime (the actual speed is decided by the external global multiplier)
PlayParticle	resPath	Loads the effect prefab (preloaded once during loading)	Read
PlayParticle	position / rotation / scale	Local transform as a child of the character	Read
PlayParticle	startTime	Instantiated when this time is reached	Read
TriggerEvent	eventName	Event dispatch (by ID)	Read; a host whose enum differs needs the mapping adapter described in 5.4
TriggerEvent	startTime	Triggers once when this time is reached	Read
TriggerEvent	spawnOffset / projectilePath	Projectile origin and flight effect	Integrated (effective when the flight effect path is non-empty — origin = character root + rotation × local offset, bullet prefab = action file; fully falls back to the Skill Table when unconfigured or loading fails)
TriggerEvent	regionShape / regionRadius / regionAngle / regionWidth / regionLength	Damage and Buff region detection	Integrated (thin adapter: tool values take precedence when the action file region is valid, otherwise falls back to the Skill Table; the formula is region radius + target hit radius)
TriggerEvent	regionOffset	AOE region center offset	Integrated (the circle center offset rotates with

			facing)
TriggerEvent	buffSectorDir / buffRectOffsetX	Enemy Buff direction and forward/backward offset	Integrated (effective when the action file region fields deviate from defaults: the first Skill Table candidate is the circle center, direction = target→hero, plus the target hit radius; old files with all-default fields use the Skill Table entirely)
TriggerEvent	hitEffectPath	Hit/Release VFX	Integrated: event 5 is the Enemy Buff Hit VFX (read first, falls back to the Buff table when missing); event 3 is the AoE Release VFX (played once at the region center, not dependent on the Buff table)
TriggerEvent	persistEffectPath	Persistent VFX	Do not read: on-device appearance is owned by the Buff table; the field is Editor/Target Range preview only
PlayAudio	All fields	In-action audio	The tool runtime does have consuming logic (AudioSource pool), but the host must register an asset loader to obtain the audio; if the host does not integrate it, in-action audio will not play in the game
Clip.active / Track.active	Enable flag	Disabled clips do not trigger	Must be respected by the host dispatcher; the Editor is self-consistent with it

5.2.1 Asset Ownership Table (Who Decides)

The same presentation may be described by both the "action file" and the "Skill/Buff table"; there must be a unique owner for each, otherwise what you see in the Target Range and what you get on device will inevitably diverge. The convention is as follows:

Content	**Owner**	**Game-side behavior**
Animation (PlayAnimation.resPath)	Action file	Read (matches the animation set by file name)
Particle effect (PlayParticle.resPath)	Action file	Read
Hit/Release VFX (TriggerEvent.hitEffectPath)	Action file	Event 5 = Hit VFX: read first, falls back to the Buff table when empty or the asset is missing; event 3 = AoE Release VFX: played once at the region center, with no Buff table fallback. Coverage granularity see 5.2.2

Persistent VFX (TriggerEvent.persistEffectPath)	Buff table	Not read; the field is only for Editor/Target Range preview
Flight effect (TriggerEvent.projectilePath + spawnOffset)	Action file first, Skill Table fallback	When projectilePath is non-empty and loads successfully, both the origin and the bullet prefab come from the action file; when unconfigured/loading fails, the Skill Table is used entirely
Enemy Buff region direction/offset (buffSectorDir / buffRectOffsetX)	Action file first, Skill Table fallback	When the action file region fields deviate from defaults, Buff targets are filtered by the tool region; old files with all-default fields still use the Skill Table
Audio (all PlayAudio fields)	Action file	Read (the host must register an asset loader and provide playback)
Detection region parameters (region*)	Action file first, Skill Table fallback	See 4.7 for the detection semantics

Every VFX slot in the Editor is annotated with an ownership hint; the Target Range always previews according to the action file, so "Hit/Release VFX" can be signed off directly in the Target Range, while "Persistent VFX" only serves to confirm timing and attach points, with the final appearance decided by the Buff table.

5.2.2 Hit VFX Coverage Granularity (Important)

A Hit VFX in an action file only covers "the one buff directly specified by that trigger event" and is not inherited downwards. Example: a skill triggers buff A, and buff A derives buff B inside the Buff table; after changing the Hit VFX of buff A in the Editor, buff B still plays the effect configured for it in the Buff table.

This is intentional: the layered presentation of secondary/derived buffs is designed by game designers, and the tool does not decide on the game's behalf whether to propagate the change.

This produces a common misunderstanding: art changes the Hit VFX in the tool, yet on device it "looks unchanged" — first confirm whether the skill also carries a derived buff; when both VFX play at the same time, the one not overridden masks the effect of the change.

The Release VFX of event 3 (Melee AoE) does not have this problem: it is not attached to any buff and is played once at the region center by the event itself, so it is naturally unaffected by the "derived buffs are not inherited" rule.

The resulting direct conclusion: if your host enum differs from the tool contract (specifically at ID 6), add the mapping described in 5.4 instead of renumbering either side. Region and VFX fallbacks resolve as follows:

AoE (event 3): uses the valid action-file region; falls back to the Skill Table otherwise.

Enemy Buff (event 5): when the action-file region fields deviate from defaults, takes the first Skill Table candidate as the center, direction target→hero, filtering targets with the same ShapeUtility

formula (including buffSectorDir / buffRectOffsetX and the target hit radius); all-default files still use the Skill Table entirely.

Projectile (event 2): reads spawnOffset for the spawn point. When projectilePath is non-empty and loads successfully, the bullet prefab comes from the action file; otherwise it fully falls back to the Skill Table.

Hit VFX: action file first, Buff table fallback.

AoE Release VFX: played by the event itself at the region center, no fallback.

Persistent VFX: the only VFX field the runtime must not read.

5.3 Time Semantics

- Forward-only advance: runtime time accumulation = frame duration × global playback multiplier; clip startTime is compared frame by frame and latched once reached, ensuring each clip triggers only once.
- End condition: accumulation reaching the action's total length length marks the end. length is therefore the only overall pacing length baseline at runtime and must not be arbitrarily rewritten externally.
- Signal clips have length 0 and their end equals the trigger moment, so "the time of the last event" is also counted into the total length; if an action has only signals and no animation, the total length is the last event time.
- Reverse playback, looping, and cross-fading are not supported; blend fields on the Editor side are not read at runtime.
- Same-time concurrency: duplicates at the same time on the same track are already prohibited by the Editor; the same time on different tracks all triggers in the same frame, iterated in Temps array order, with stability not guaranteed.

5.4 Event ID Contract (Key Integration Point)

- IDs written by the tool (`Runtime/Directables/OptionParams.cs`): 1 Melee Single-Target (Hit), 2 Projectile (EmitBullet), 3 Melee AoE (Aoe), 4 Instant Laser (InstantSpell), 5 Enemy Buff (AddBuff), 6 Self Buff. Compact and continuous, stable and unchanged — this is the tool's formal external contract.
- Recommended host enum: mirror the same six IDs. Some existing projects already put a different event at ID 6 (for example a buff-mapping entry) and place Self Buff at 8.
- Conflict point: ID 6 means Self Buff in the tool. Never renumber the tool contract to match a legacy host enum.
- Recommended integration (the tool side stays self-consistent and unchanged; the host adds a thin adapter): add one mapping line at the event dispatch entry — pass IDs 1–5 through unchanged and translate the tool's 6 to the host's Self Buff entry (8 in the example above). No host enum changes, no tool contract changes, and existing action files need no migration. Put the mapping table at the dispatch entry of your skill action component, before the enum validity check.

IDs are persisted as bare integers, with no namespace or version concept. When adding or removing events in the tool contract, connect through the mapping table; do not modify the host enum directly, to avoid one change affecting the whole system.

5.5 Shared Pure Functions for Shape Detection

- The tool side exposes the ShapeUtility static class from `Runtime/ShapeUtility.cs`, providing three public pure functions so the Target Range and the host can share the same detection logic:

Function	Applicable event	Core parameters
IsPointInShape(targetPos, centerPos, forward, regionShape, regionRadius, regionAngle, regionWidth, regionLength, targetHitRadius)	Melee AoE (Aoe), directly usable for Enemy Buff	Shape center, forward reference, shape parameters, target hit radius
IsPointInBuffRect(targetPos, centerPos, toHero, rectOffX, regionWidth, regionLength, targetHitRadius)	Enemy AddBuff rectangle	Target→hero direction axis, rectangle center forward/backward offset, target hit radius
IsPointInBuffSector(targetPos, centerPos, toHero, regionRadius, buffSectorDir, targetHitRadius)	Enemy AddBuff sector	Target→hero direction axis, sector rotation, target hit radius

- Target hit radius (targetHitRadius, default 0): detection is computed with "the target having volume" — for a circle/sector it is distance ≤ region radius + hit radius, for a rectangle each of the four sides is expanded by the hit radius, allowing slightly behind the rectangle. The game's on-device detection is exactly radius + GetHitRange(); the Target Range must pass the dummy's Hit Radius to match the device.
- The Target Range dummy (TargetRangeDummy) Hit Radius is a configured value (default 0.5 m), written by the Target Range config per prefab; it must equal the hit_range value of that unit on device, otherwise the preview hit count will be lower than on device.
- Hit Radius configured per prefab: the Target Range config file's `defaultHitRadius` (global default, default 0.5 m) and the `hitRadiusConfigs` list (prefabPath + hitRadius, with JsonUtility simulating a map through an entry list) persist in the Target Range config file (Storage Path/Target Range Name.json); when a prefab path is given and the list matches, the list value is used, otherwise the global default. Instance effective Hit Radius = configured value × instance scale factor (mean of the horizontal X/Z components of lossyScale). A Hit Radius slider (0–5 m, step 0.05) has been added directly below the three "Target Dummy Formation" buttons in the dynamic area; dragging updates on-scene dummies in real time, and releasing the mouse writes back to the Target Range config; switching prefabs or opening a different target range shows the value already configured for that prefab. Melee AoE and Enemy Buff detection uniformly pass the effective radius; the orange range frame is the intended design range and the light-cyan ring at each dummy's feet is the hit volume — intersection means a hit, and no expanded dashed line is drawn any more, avoiding confusion between two "range" semantics.
- Target dummy scale slider: directly below the Hit Radius slider in the dynamic area, multiplier 0.5–3.0, step 0.05, default 1.00, stored in the Target Range config as `dummyScale`; the Hit Radius grows automatically with the scale, so a small mob can be enlarged into an elite.

- At runtime each dummy shows its own hit ring at its feet (light-cyan thin line, fixed 48 segments, radius = effective Hit Radius including scale, raised 0.01 m above the ground to avoid Z-fighting), changing in real time while the slider is dragged and hidden when not running; the orange range frame is the intended design range and the light-cyan ring is the hit volume — intersection means a hit (no expanded dashed line is drawn any more).
- All three functions are static, depend only on Vector3 / Mathf, and have no stateful side effects; they can be called directly at any runtime, depending neither on the Editor assembly nor on AssetDatabase.
- If the game side wants on-device detection to match the Target Range, it only needs to read the TriggerEvent.region* fields and call the functions above when dispatching Aoe / AddBuff events; if not integrated, the Skill Table range continues to be used, and JsonUtility automatically ignores unread fields, leaving legacy logic completely untouched.

5.6 Asset Paths and Loading

- All asset fields are Assets-relative path strings (e.g. prefabs or animations under Assets/Bundles/...).
- Inside the tool, every "load asset by path" (animation clips, effect prefabs, audio, character prefabs) goes through `StagecraftResources.LoadAsset<T>(path)`: in the Editor (Editor preview and Target Range Play) it defaults to AssetDatabase when no loader is registered, with the same behavior as before; a packaged game must call `StagecraftResources.SetLoader(implementation)` to register the host's own loader (Addressables / YooAsset / Resources all work), otherwise assets cannot be loaded, the corresponding clips are deemed invalid, and they are skipped silently.
- Animation assets are matched on the game side by file name (without extension) against the character animation library, so animation names must match the character animation set; the path itself does not participate in playback.
- Effect prefabs and bullet prefabs are given by the Skill Table on the game side; same-named paths in the action are used for Editor preview and the Target Range. When the two are inconsistent, "what the Editor shows is not what the device gets".
- Audio: the tool runtime already implements PlayAudio consumption (AudioSource pool, supporting volume gain, speed change, fadeIn/fadeOut). After a loader is registered, this logic can be reused on device directly; the host only needs to ensure audio clips can be loaded by path; if the host chooses not to consume in-action audio, it should state explicitly that "in-action audio is for Editor/Target Range preview only, and the device side is handled by the Skill Table or an audio middleware".
- The action file itself is pointed to by the action address field of the Skill Table, and is asynchronously loaded as a text asset and then deserialized.

5.7 Data the Runtime Should Ignore

- The entire editorData section: rolePrefabPath, projectilePreviews, aoePreviews, laserPreviews, buffPreviews, actionPreviews. By design it is a black box to the runtime.
- TriggerEvent.clipId (Editor-internal pairing only), viewTimeMin / viewTimeMax (Editor view range only), formatVersion (migration only).

- Legacy fields buffRadius, buffOffset, skillBuffOffset: always at default values in new data; ignore them even if read.

5.8 Forward and Backward Compatibility

- Adding fields is safe: old readers ignore unknown keys, so appending contract fields does not break older parsing; new readers reading old files take constructor defaults for missing keys, then correction is applied by migration logic (old buffRadius migrates to regionRadius, and the legacy Enemy Buff 90° is normalized to 360°).
- Reordering IDs is unsafe: adding/removing or reordering event IDs cannot be covered by "ignore unknown keys" — old actions show as unconfigured, and new actions execute misaligned in old runtimes. Such changes must be paired with version gating or an explicit mapping table.
- Empty tracks are stripped on save: written files contain no zero-clip tracks, so readers do not need to support the "empty track" shape.

5.9 Integration Checklist

No.	Checklist item	Pass criteria
1	First-line type row	Correctly ignored or validated; not part of JSON parsing
2	Temps recursive restoration	Group/Track/Clip three layers restorable by type name, and the types are not stripped
3	Validate call	length recomputed after restoration; time detection matches the Editor
4	Event ID alignment	Tool constants match the runtime enum item by item (especially 6)
5	Region field integration	If integrated, the detection formula matches the Editor/Target Range; if not, the documentation states explicitly that the Skill Table is authoritative
6	active filtering	Disabled tracks/clips do not trigger
7	Asset paths	Assets-relative paths resolvable by the loader; animation names match the character animation set
8	Encoding and version	UTF-8 without BOM; formatVersion not higher than the supported version
9	Time semantics	Forward-only advance, signal ends counted into the total length, total length minimum 0.1 s
10	Audio	If in-action audio is needed, the host must call `StagecraftResources.SetLoader` to provide a loader and integrate AudioSource playback (the tool runtime already provides this logic)

6. Appendix

6.1 Default Values Quick Reference

Item	Default value
Editor view range	0–5 s (minimum span 0.25, maximum 240)
Magnetic snapping	On (10 px radius, Shift temporarily disables)
Spawn offset spawnOffset	(0, 1, 0)
Region radius regionRadius	3 m
Sector aperture regionAngle	360° (full circle)
Rectangle width regionWidth / length regionLength	2 m / 4 m
Sector direction buffSectorDir / rectangle offset buffRectOffsetX	0 / 0
Preview projectile speed / range / parabola height	3 m/s / 10 m / 2 m
Laser preview length / origin offset	6 m / (0, 1.5, 0)
Hit point hitOffset / persist point persistOffset	(0, 1, 2) / (0, 1.8, 0)
Tremble / Screen Shake / Flash	0.03 m · 0.2 s / 0.15 m · 0.25 s / white · 0.3 s
Volume / Playback speed	1 / 1
Camera offset cameraOffset	(0, 30, -7), Y≥0
Viewport aspect ratio	1080×1920
Attack range attackRange	1 m (clamped to 1–6 on load)
Move speed moveSpeed	1 m/s (0.1–20)
Spacing distance dummySpacing	2 m (0.5–10)
Laser refraction laserRefraction	On
Render pipeline pipelineMode	Auto-detect
Invocation item limit / skill button limit	30 / 8

6.2 Paths Quick Reference

Content	Location
Action file	Under the user-configured "Action save path", *.action.bytes (JSON)
Target range config	Assets/Bundles/Stagecraft_TargetRange/Ranges/Target Range Name.json
Legacy target range config (auto-migrated)	Assets/Bundles/Stagecraft_TargetRange/Config/target_range.json → Ranges/Default Target Range.json
Editor preferences	Editor/ActionEditor.txt inside the package
Event type config	Editor/Config/SignalConfig.json inside the package
Smoke report	Project root ae_range_smoke.txt (triggered by ae_range_smoke.flag)
Sign-off Log	Project root StagecraftVerify/records.jsonl (one JSON per line, bound by the action file hash; outside Assets, so it is not packed into the build)
Sample actions	Samples~/Demo Actions inside the package (import via Package Manager)

Package layout (one level below the top folders):

```
Runtime/   Assets/  Directables/  TargetRange/

Editor/    Config/  Languages/  Preview/  Properties/  Resources/  Samples/  TargetRange/
Undo/  View/  Window/

Tests/     EditMode test assembly
```

```
Documentation~/      this guide (.txt and .docx), EULA.txt

Samples~/  Demo Actions/ (two sample action files + README.md + Demo Assets/)
```

6.3 Capability Boundaries

These are real limits of the shipped 1.0.0. None of them restrict the Action Editor itself — they apply to the bundled Target Range scene, to fields the runtime does not read, or to the render pipeline.

Area	Limit	Impact
Target Range — skill buttons	At most **8** invocation buttons on the runtime panel	Buttons beyond the eighth cannot be shown; the action list still holds up to 30
Target Range — action import	At most **30** actions per range config (adds are rejected as a whole when the total would exceed it)	Group different actions into separate range configs
Target Range — panel-only sliders	`enemyBuffHeight`, `regionCenterMode`, `regionForwardDistance` are stored in the range config and shown on the panel, but the event detection path does not consume them	They only affect panel display; the actual region comes from the event clip's `region*` fields
Runtime — `playbackSpeed`	Written to the action file but **not read** by the runtime	Global playback speed comes from the game, not from each clip
Runtime — `persistEffectPath`	Written to the action file for editor/Target Range preview only; the runtime should **not** read it	Persistent buff visuals are owned by the Buff table
Render pipeline	The Target Range scene does not run under the Built-in Render Pipeline	See 1.1 for the per-pipeline support table
Target Range — Dummy visuals	When no Dummy prefab is assigned, the scene spawns a placeholder capsule with a Built-in shader	Under HDRP assign real prefabs, otherwise the placeholder renders magenta

6.4 Glossary

Term	Meaning
Action	One skill presentation: a timeline plus a set of tracks, saved as a single `.action.bytes` file
Timeline	The horizontal editing surface; time runs left to right, the playhead marks the current time
Track	A horizontal lane holding clips of one kind (animation, signal, effect, or audio)
Clip	A single item on a track with a start time and a length
Signal track	The track that holds gameplay events (the "Event" clips)
Event clip	A clip that records when a gameplay event fires and the contract data the runtime reads
Animation track	The track that plays a character animation
Effect track	The track that plays particle or VFX prefabs
Audio track	The track that plays audio
Asset List	The window that scans the save path and lists every action file with its warnings and signals
Inspector	The right-hand panel that edits the selected clip's fields
Playhead	The vertical marker indicating the current preview time
Target Range	The 3D scene window used to play an action against

	target dummies and verify event detection
Target Dummy	A stand-in enemy in the Target Range
Hit Radius	The per-prefab radius used when deciding whether a dummy is inside a region
Melee Single-Target	Event 1 — one damage instance on the locked target
Melee AoE	Event 3 — one damage instance per target inside the region
Instant Laser	Event 4 — an instantaneous beam hit test
Enemy Buff	Event 5 — a buff applied to enemies inside the region
Self Buff	Event 6 — a buff applied to the character
Region	The shape (sector/circle or rectangle) used to select targets; defined by `regionShape`, `regionRadius`, `regionAngle`, `regionWidth`, `regionLength`, `regionOffset`
Release VFX	The effect played once when an event fires
Hit VFX / Persistent VFX	Buff visuals: played once on hit, or kept looping while the buff lasts
Projectile	A spawned flying effect with a speed and an attach point
Spawn Point	The offset where a projectile or effect originates, in character-local meters
Attach Point / Bone	The bone an effect is attached to when it follows the character
Padding	Extra distance added to the hit test so effects that visually touch the target still register
Smoke Test	An automated Target Range run that writes a report file instead of requiring manual clicking
Sign-off Log	The JSONL record of review results, bound to the action file hash
Reviewer / Result / Notes	Sign-off fields: who reviewed, whether it passed, and free-form remarks
Pass / Fail / Signed Off / Failed / Stale / Not Signed Off	Sign-off states; **Stale** means the action file changed after the last review
Effect Group	The grouping container inside an action that holds the tracks

6.5 Screenshot Checklist (Store Listing and Manual)

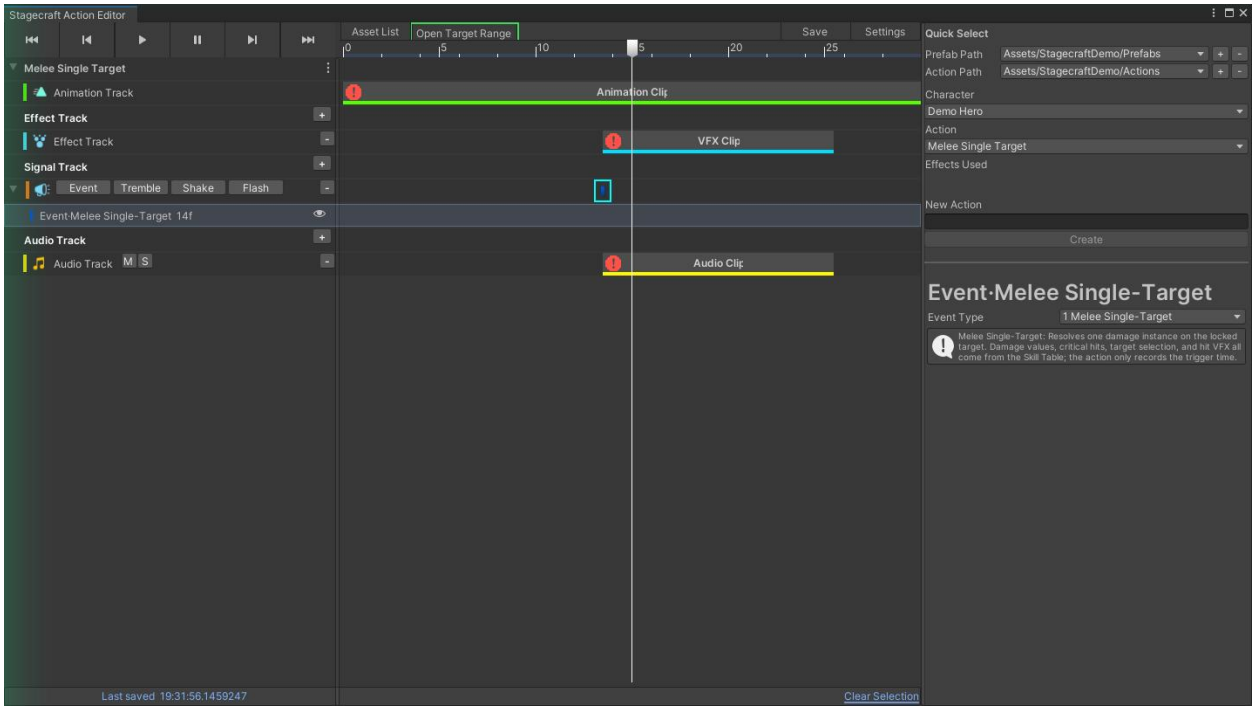
Shooting rules first: use the dark editor skin, a window at least 1440 px wide, and capture once at 100% Windows scaling and once at 125%; no personal project asset names may appear in frame.

#	Shot	Required state	Setup data
1	Action Editor overview	One action open, 4 tracks populated, playhead mid-clip	Use `Melee Single Target` sample, open the Inspector on an event clip
2	Signal track close-up	Event clips in a row, all button labels fully visible	Any action with 4+ signal clips; intended to show button sizing
3	Event Inspector	Melee AoE selected, region fields visible	Sample action, event 3
4	Asset List	Two actions listed, warning column empty, signal column filled	Import both samples
5	Target Range static area	Dummy prefab assigned, Hit Radius and formation visible	Placeholder capsule is acceptable but assign a real dummy for the store

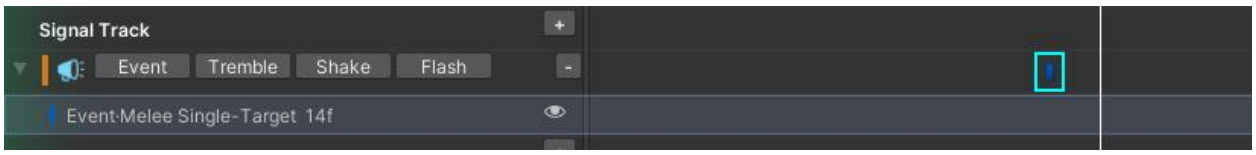
			shot
6	Target Range running	Hit rings lit on 2 of 3 dummies, orange frame visible	Triangle formation, Melee AoE event at 0.6 s
7	Sign-off Log	One record with Reviewer and Notes filled	Run once, then sign off

Screenshots

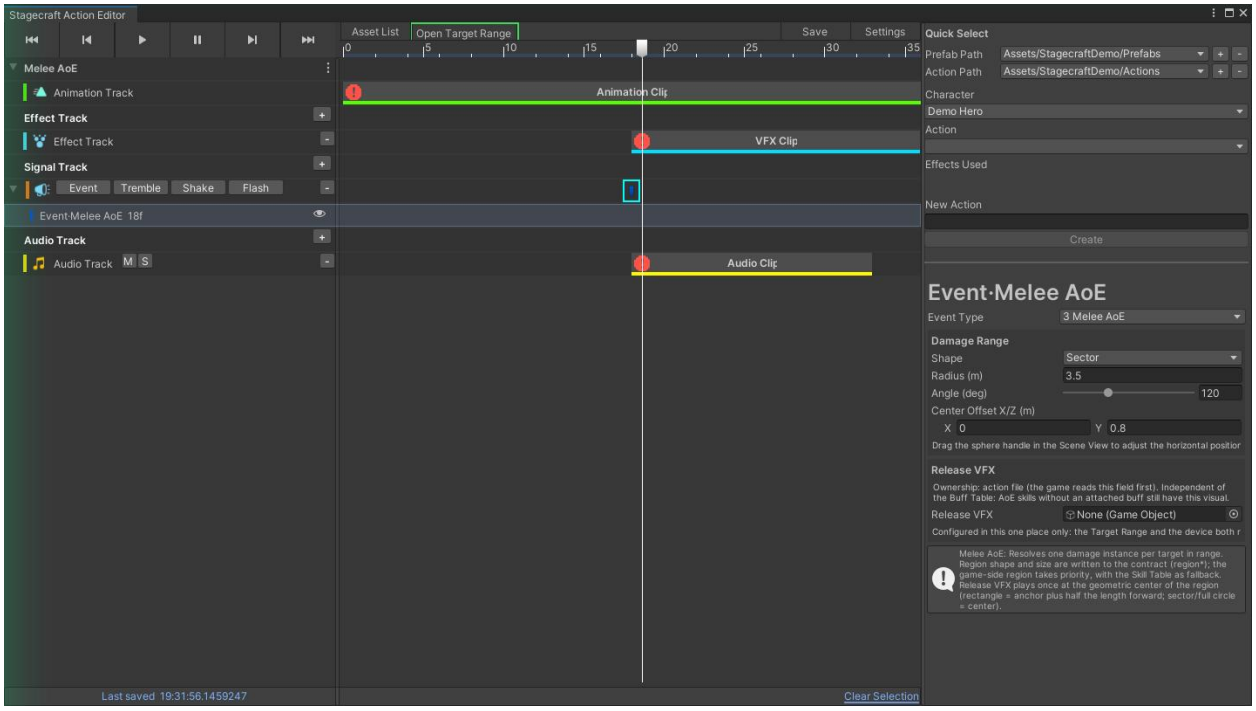
Reference images used by this manual. The same files are used for the store listing.



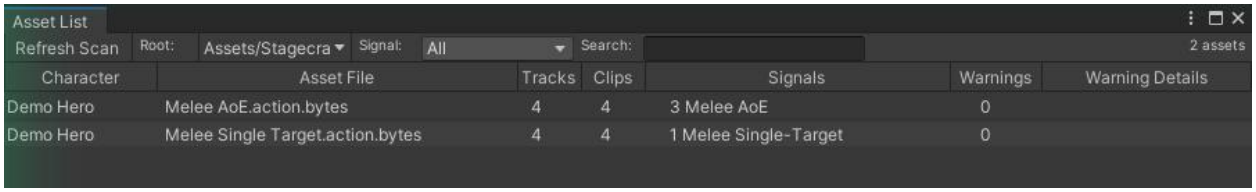
1 - Action Editor overview



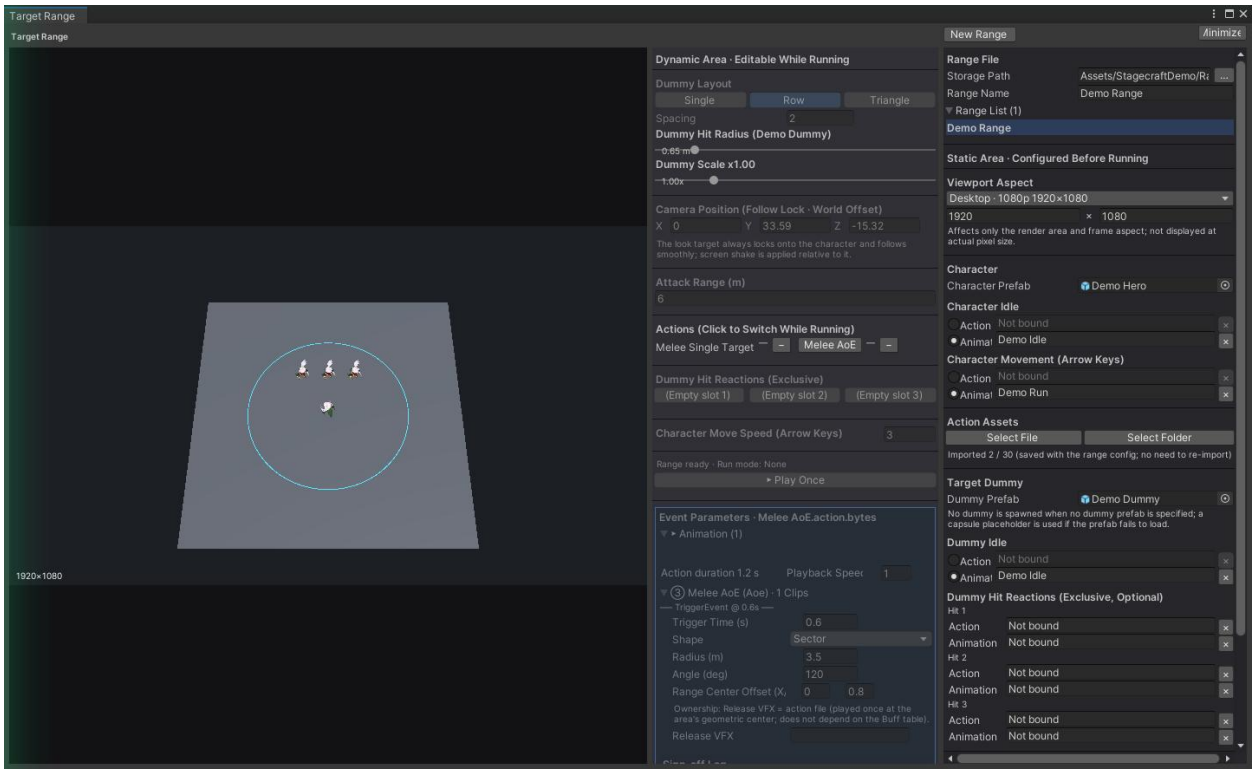
2 - Signal track close-up



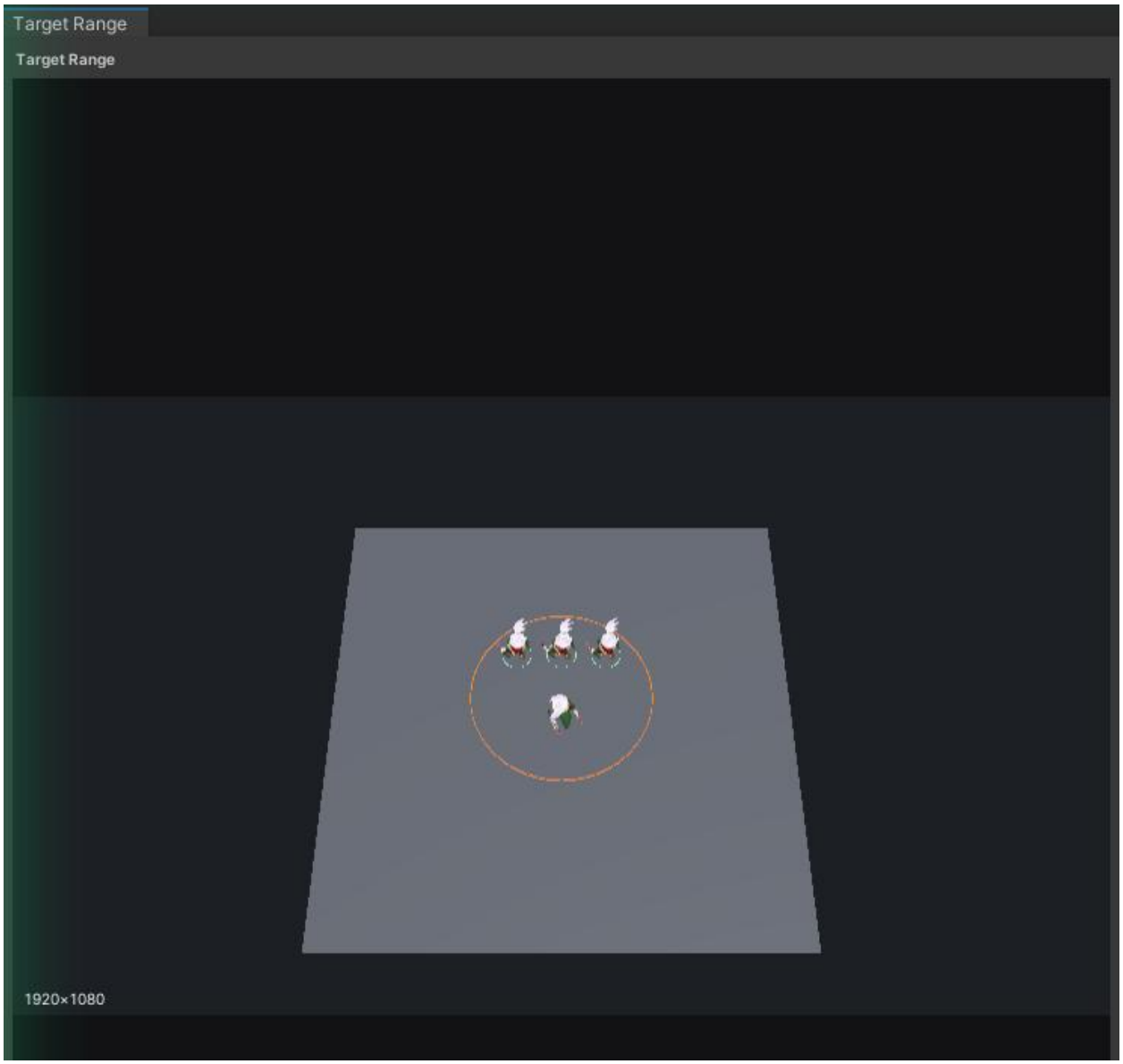
3 - Event Inspector



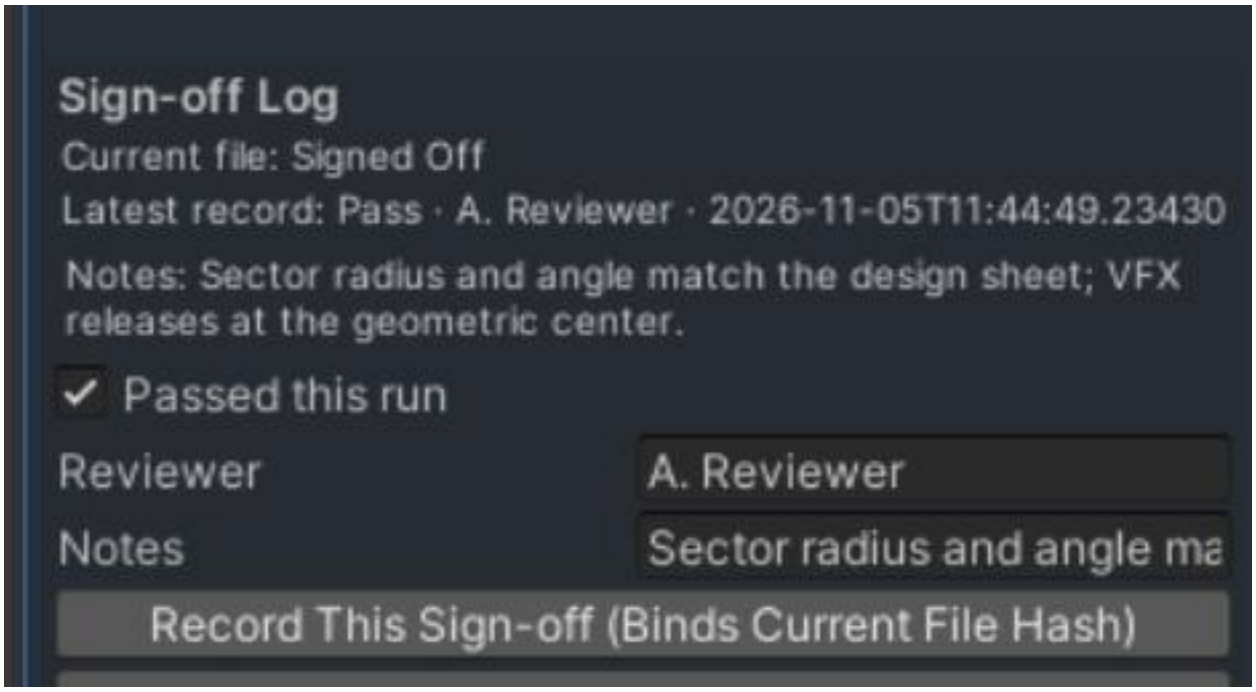
4 - Asset List



5 - Target Range static area



6 - Target Range running



7 - Sign-off Log

Seven shots are enough for the store listing and this manual; the list above is the complete set.

Shot 2 is the one that proves the UI does not clip translated labels; keep the same window width in every shot so reviewers can compare.